SISTEMAS DISTRIBUÍDOS · CEFET-MG · 2026/2

Transferência de arquivos Peer-to-Peer

Trabalho Prático 2 · Professora Michelle Hanne · 18/10/2026

Código-fonte: https://github.com/LucasAbrandao/sd-tp2-p2p

Repositório privado. O acesso ao código para avaliação precisa ser concedido pelo proprietário.

1. Objetivo e decisões de projeto

Foi implementado um sistema de transferência por blocos em que cada processo é, simultaneamente, cliente e servidor. Um seeder inicial possui o arquivo completo; os demais peers solicitam blocos a vizinhos predefinidos e compartilham os blocos já verificados, mesmo com o download incompleto.

A linguagem escolhida foi Python 3.10 ou superior. O programa usa somente a biblioteca padrão: asyncio para concorrência e sockets TCP não bloqueantes, json para serialização, hashlib para SHA-256 e pathlib para armazenamento. Não há tracker nem serviço central; o manifesto e a lista estática de vizinhos são fornecidos antes de iniciar os processos.

Arquitetura de um peer

Componente	Responsabilidade
Servidor TCP	Aceitar conexões, anunciar o inventário e atender pedidos de blocos disponíveis.
Clientes concorrentes	Manter uma conexão por vizinho, consultar seu inventário e solicitar blocos ausentes.
Armazenamento	Validar, persistir, recuperar e remontar o conteúdo na ordem dos índices.
Registro JSONL	Identificar peer, evento, índice, origem/destino e progresso de cada bloco.

O listener começa antes dos clientes. Cada vizinho recebe uma tarefa própria; uma reserva local evita que dois clientes do mesmo processo solicitem o mesmo bloco em paralelo. A escolha percorre índices em ordem circular, com ponto inicial derivado do identificador do peer, e atualiza o inventário remoto a cada lote de até 32 pedidos. Essa regra é simples e não pretende otimizar raridade.

Leitura, gravação e montagem são enviadas a threads de trabalho para liberar o laço de eventos durante operações de disco. Após concluir, o peer continua atendendo conexões até ser encerrado.

2. Metadados, protocolo e integridade

O manifesto é um arquivo JSON com exatamente cinco campos. Todos os peers usam a mesma cópia. O identificador file_id é o SHA-256 do JSON canônico desse manifesto, incluindo nome, tamanho, fragmentação e hashes. A conexão só prossegue quando versão e file_id coincidem.

Campo	Conteúdo
name	Nome de arquivo seguro, sem diretórios.
size / block_size	Tamanho total e tamanho fixo dos blocos, em bytes.
sha256	Hash do arquivo inteiro.
blocks	Lista ordenada de hashes SHA-256, um por bloco; o último pode ser menor.

Cada mensagem é um quadro: um inteiro sem sinal de 4 bytes em ordem de rede (big-endian) com o comprimento do corpo, seguido do objeto JSON em UTF-8. readexactly trata cabeçalhos e corpos fragmentados pelo TCP. Quadros vazios ou acima de 2 MiB são rejeitados; leitura e escrita têm timeout.

Mensagem	Papel
hello	Handshake com versão 1, identificador do peer e file_id.
status	Solicitar ou responder a lista de índices atualmente disponíveis.
get	Solicitar um bloco pelo seu índice, começando em zero.
block / missing	Responder dados em Base64 ou informar que o bloco não está disponível.

Persistência e recuperação

Um bloco recebido precisa ter índice, tamanho e SHA-256 corretos. Ele é escrito em um arquivo temporário e renomeado antes de entrar no inventário. Blocos ficam em .blocks/<file_id>/<índice>.part. As gravações por bloco usam o cache do sistema operacional; não há fsync por KiB. Na reabertura, cada bloco é revalidado, e conteúdo incompleto ou corrompido não é anunciado.

A montagem percorre todos os índices, confere novamente cada bloco, tamanho total e SHA-256 final. As leituras usam até oito threads, em lotes de até 64 blocos e no máximo 4 MiB; a escrita segue a ordem original. O arquivo pronto é sincronizado e publicado atomicamente com um hard link, sem substituir um destino de conteúdo diferente. Os testes locais usam NTFS; o destino precisa permitir hard links. Blocos são mantidos após a montagem para continuar compartilhando e permitir retomada.

Se um vizinho falha, a reserva é liberada e a tarefa tenta reconectar. Blocos íntegros já salvos não são baixados de novo. Uma mensagem inválida encerra somente sua conexão. O prazo total do download evita espera ilimitada quando nenhum vizinho tem os blocos restantes.

3. Estudos de caso e resultados medidos

Foram executadas as 24 combinações de 2/4 peers, blocos de 1024/4096 bytes e arquivos de 10/20 KiB, 1/5 MiB e 10/20 MiB. Adotaram-se unidades binárias: 1 KiB = 1024 bytes e 1 MiB = 1048576 bytes. Cada arquivo foi gerado com os.urandom, e cada peer foi um processo separado, com comunicação TCP em 127.0.0.1, no Windows, Python 3.13. Uma execução por combinação.

Com dois peers, A e B são vizinhos e A é a fonte. Com quatro, A conhece B/C; B e C conhecem A/D; D conhece apenas B/C. Os receptores iniciam antes de A. Essa topologia obriga D a obter conteúdo de intermediários e permite verificar o compartilhamento antes de eles terminarem de baixar.

Nos casos de quatro peers e arquivos de até 20 KiB foi aplicado um atraso de 3 ms por bloco, registrado no resultado, para observar o compartilhamento parcial sem depender somente do escalonador. Os demais casos não usam atraso artificial. Portanto, as comparações não isolam apenas o número de peers.

Arquivo	2 peers / 1 KiB	2 peers / 4 KiB	4 peers / 1 KiB	4 peers / 4 KiB
10 KiB	0.861 s	0.811 s	1.643 s	1.604 s
20 KiB	0.877 s	0.853 s	1.853 s	1.544 s
1 MiB	5.855 s	1.861 s	7.523 s	3.000 s
5 MiB	28.298 s	7.018 s	37.220 s	9.307 s
10 MiB	57.393 s	14.395 s	86.187 s	18.322 s
20 MiB	119.491 s	28.618 s	206.628 s	38.916 s

Tempo de parede desde o início do primeiro processo até a conclusão de todos os receptores, incluindo inicialização e montagem; exclui geração do arquivo e do manifesto. Os valores são observações locais, não médias estatísticas.

Resultado: 24/24 cenários aprovados. Foram verificadas 72 cópias (incluindo fontes) diretamente no sistema de arquivos: o tamanho e o SHA-256 coincidiram com o original. Nos 12 cenários com quatro peers, D recebeu somente de B/C; foram registrados 26227 envios para D enquanto o intermediário ainda tinha apenas parte do arquivo.

Exemplo real do registro de compartilhamento parcial:

```
{"peer":"B","event":"block_sent","index":5,"target":"D","have":3,"total":10}
```

evidence/results.csv contém todos os tempos; results.json inclui hashes, fontes, contagens e parâmetros; selected-events.jsonl conserva amostras reais dos logs. Os logs completos e os arquivos de trabalho ficam em .runtime, sem entrar no Git.

4. Avaliação e reprodução

A verificação de conteúdo foi bem-sucedida nos tamanhos pequenos, médios e grandes. Para 20 MiB, os tempos foram 119.491 s e 28.618 s com dois peers, e 206.628 s e 38.916 s com quatro peers (blocos de 1 e 4 KiB, respectivamente). Blocos de 4 KiB reduzem de 20480 para 5120 os blocos necessários para 20 MiB, diminuindo pedidos, hashes, registros e operações de arquivo.

O teste com quatro processos também transfere mais cópias e inclui o caminho por intermediários. Disco, cache, escalonamento, Base64 e inicialização influenciam os tempos. Uma execução em loopback com duas topologias distintas não sustenta uma conclusão geral sobre escalabilidade. Seriam necessárias repetições e condições de rede controladas para separar esses efeitos.

Testes complementares

A suíte local de 28 testes passou. Ela cobre serialização e validação do manifesto; seed inválido; blocos com tamanho ou hash incorretos; duplicatas; retomada com corrupção; destino existente; arquivo vazio e último bloco parcial. Para a rede, cobre quadros fragmentados, concatenados e truncados, JSON/UTF-8 inválido, limites, timeout, bloco ausente, índice inválido e listener ativo após uma conexão malformada.

Quatro testes iniciam processos reais e verificam o resultado: dois peers com último bloco parcial, arquivo menor que um bloco, quatro peers compartilhando antes da conclusão e chegada tardia do seed. O último aguarda um erro de conexão observado antes de iniciar a fonte, comprovando a nova tentativa. O fluxo de CI foi incluído para Python 3.10/3.12 em Windows/Linux; esta avaliação registra a execução local.

Como executar

```
python scripts/prepare_demo.py --peers 4

python -m unittest discover -s tests -v

python scripts/benchmark.py --timeout 600
```

O primeiro comando gera arquivo, manifesto e configurações e imprime um comando para cada terminal. O README descreve a execução com dois peers, os caminhos relativos, as portas e o uso em máquinas distintas. O benchmark encerra seus subprocessos automaticamente e escreve as evidências.

Limites assumidos

O sistema atende a um cenário didático com manifesto confiável: não oferece autenticação, TLS, descoberta dinâmica ou proteção completa contra participantes maliciosos. Os limites são 128 MiB por arquivo, 1 MiB por bloco e 131072 blocos. A rede não autentica quem fornece o manifesto. Como evolução, cabem inventários compactos, seleção por raridade, armazenamento em um único arquivo e experimentos repetidos em rede física com latência e perdas controladas.